

Federación de Contexto de Lectura Primero y Control por Rol para Sistemas Multiagente Basados en LLM: una Evaluación Multi-Modelo Controlada por Equidad Frente al Paso de Mensajes Punto a Punto

Adrian Issac Mamani Quispe*, Ryan Fabian Valdivia Segovia*,
Karim Guevara Puente de la Vega*, Norka Norali Bedregal Alpaca*

*Universidad Nacional de San Agustín de Arequipa, Arequipa, Perú
admamaniq@unsa.edu.pe, rvaldiviase@unsa.edu.pe,
kguevarap@unsa.edu.pe, nbedregal@unsa.edu.pe

Resumen—Los sistemas multiagente basados en modelos de lenguaje grandes (LLM) suelen depender del paso de mensajes punto a punto, lo que puede aumentar la complejidad de integración y exige mecanismos adicionales de control de acceso granular y trazabilidad. Este estudio evalúa el Graph Context Protocol (GCP), un enfoque de federación de contexto de lectura primero que integra control de acceso por rol y por capacidad con procedencia nativa por cada lectura. GCP se compara con una implementación base de Agent2Agent (A2A) bajo un diseño controlado en el que la lógica del agente, los modelos, las tareas y los criterios de evaluación se mantienen constantes mientras solo cambia la capa de interoperabilidad. La evaluación cubre cinco escenarios usando tres modelos de peso abierto alojados localmente y tres variantes de Gemini 2.5 alojadas en la nube. Los resultados estructurales muestran que GCP reduce el crecimiento de aristas de integración de $\mathcal{O}(n^2)$ a $\mathcal{O}(n)$ y mantiene un sobrecosto de descubrimiento y de descripción de herramientas por solicitud de $\mathcal{O}(1)$, frente a $\mathcal{O}(n)$ en A2A base. En los escenarios de delegación no autorizada y de acceso forzado, GCP denegó toda operación no autorizada intentada, mientras que A2A base permitió dichas operaciones siempre que fueron intentadas; en los modelos alojados más fuertes, las lecturas no autorizadas además expusieron contenido canario confidencial bajo A2A, mientras GCP denegó el acceso. GCP también provee procedencia por lectura por construcción, mientras que una trazabilidad equivalente requiere instrumentación adicional en A2A base. Los experimentos conductuales indican éxito de tarea y costo de comunicación comparables, mientras que los experimentos canario no adversariales no muestran ventaja de privacidad entre enfoques. En conjunto, los resultados respaldan los beneficios estructurales y de gobernanza de GCP, independientes del modelo, mientras se posicionan los hallazgos conductuales como preliminares.

Palabras clave—federación de contexto; sistemas multiagente; modelos de lenguaje grandes; control de acceso; mínimo privilegio; procedencia; contención de agentes; acoplamiento de software; protocolos de comunicación entre agentes

I. INTRODUCCIÓN

El patrón de coordinación dominante para los sistemas multiagente basados en LLM es el paso de mensajes directo. Marcos como AutoGen [1], ChatDev [2] y MetaGPT [3]

componen aplicaciones a partir de agentes que envían y reciben mensajes, y el protocolo Agent2Agent (A2A) [4] estandariza exactamente esto: un agente cliente anuncia y delega tareas a agentes remotos con nombre mediante JSON-RPC. Este paradigma descende de los lenguajes clásicos de comunicación entre agentes KQML [5] y FIPA-ACL [6], que ya asumían conexiones por pares y protocolos compartidos y pre-negociados entre cada par que interactúa.

El paso de mensajes tiene un costo estructural que la literatura de ingeniería de software conoce desde hace cincuenta años. Conectar n componentes por pares requiere del orden de n^2 enlaces; un intermediario compartido al que cada componente se conecta una sola vez reduce esto a n . Este es el “espagueti de integración” que el catálogo de Enterprise Integration Patterns [7] aborda con brokers y hubs, y que la teoría del acoplamiento [8–10] formaliza: la modificabilidad está gobernada por el número y la fuerza de las dependencias entre componentes.

Estudiamos un sustrato alternativo, el Graph Context Protocol (GCP). GCP es *de lectura primero*: un agente obtiene lo que necesita leyendo contexto desde nodos de conocimiento de propiedad independiente en lugar de recibir mensajes empujados. Es *controlado por rol*: cada lectura pasa por una compuerta de política que autoriza la solicitud según el rol y las capacidades requeridas, denegando por defecto [11, 12]. Y es *auditado*: cada decisión de lectura o delegación —conceder o denegar— produce un registro de procedencia estructurado.

a) *Tesis (forma medida)*:. A igual éxito de tarea, la federación de contexto de lectura primero y controlada por rol ofrece ventajas *estructurales* sobre el paso de mensajes punto a punto en acoplamiento y trazabilidad, y evidencia *preliminar* de costo menor o igual bajo ciertos escenarios. Presentamos las afirmaciones estructurales como fuertes (son deterministas o arquitectónicas) y las conductuales como preliminares (modelos de peso abierto, pocas semillas).

b) *Alcance de la comparación.*: Somos explícitos sobre lo que representa el brazo A2A. No afirmamos que el paso de mensajes *no pueda* auditarse: registros, middleware, proxies o gateways pueden añadir un rastro de acceso. Nuestra comparación es contra A2A *base* —el protocolo tal como se distribuye, sin dicha instrumentación— y su propósito es mostrar que GCP convierte la procedencia y el control por rol en *primitivas nativas* en lugar de extensiones opcionales. Donde resulta útil discutimos un segundo punto de referencia, A2A aumentado con registro mínimo de lecturas (Sección IV).

c) *Contribuciones.*:

- 1) **Arquitectura y una especificación reproducible.** Describimos GCP (de lectura primero, controlado por rol, auditado, con una compuerta de delegación de capacidad más estricta) con precisión suficiente para reimplementarlo: formato de consulta, estructura de nodo y política, credencial de rol, registro de procedencia y el algoritmo de autorización (Sección III).
- 2) **Un método de benchmark controlado por equidad.** Una única definición de escenario y un único modelo inyectado construyen ambos brazos; solo cambia la capa de interoperabilidad (Sección V).
- 3) **Una comparación empírica multi-modelo** sobre cinco escenarios —incluyendo una prueba de *acceso forzado* que apunta directamente al nodo confidencial— y cuatro familias de modelos (tres de peso abierto, una familia alojada de frontera) en siete configuraciones de ejecución, separando resultados deterministas de resultados dependientes del modelo (Sección VI).
- 4) **Definiciones de métricas** para acoplamiento estructural, sobrecosto de descubrimiento/prompt de herramientas, una taxonomía de fuga/agencia de cuatro vías, completitud de procedencia y contención de delegación/acceso forzado (Sección IV).

II. ANTECEDENTES Y TRABAJO RELACIONADO

A. El paso de mensajes como línea base multiagente

Los lenguajes de comunicación entre agentes establecieron el paradigma de paso de mensajes: KQML [5] envolvía cargas útiles en performativos tipados sobre conexiones por pares, y FIPA-ACL [6] estandarizó un mensaje en el que emisor y receptor deben compartir lenguaje de contenido y ontología antes de que cualquier intercambio tenga éxito. Los marcos LLM modernos reinventaron esto en lenguaje natural. AutoGen [1] programa aplicaciones como conversaciones; ChatDev [2] organiza pares de roles en una “cadena de chat” fija; los estudios de revisión [13] clasifican la comunicación en variantes de igual a igual, centralizada, en capas y de conjunto compartido. El Model Context Protocol [14] estandariza el acceso a contexto de agente a herramienta (una forma temprana de lectura primero), mientras que A2A [4] estandariza la delegación de tareas de agente a agente —la línea base contra la que hacemos benchmark. Zhang et al. [15] reportan un 28–73% de tokens excedentes en grafos de comunicación entre agentes y recuperan la mayoría podando la topología; Mao and

Mirhoseini [16] reemplazan tanto la orquestación centralizada como el enrutamiento de igual a igual por un contexto verificado compartido a aproximadamente la mitad del costo. Nosotros añadimos autorización controlada por rol, procedencia auditada por lectura, una compuerta de contención de agencia y una comparación multi-modelo controlada por equidad.

B. El acoplamiento y el problema de n^2

Que menos enlaces entre componentes produzcan sistemas más mantenibles es uno de los resultados más antiguos de la ingeniería de software. Parnas [8] mostró que la descomposición por ocultamiento de información produce un acoplamiento mucho menor que el particionamiento estilo diagrama de flujo; Stevens et al. [9] hizo medibles el acoplamiento y la cohesión; Briand et al. [10] dio un marco teórico de grafos en el que el acoplamiento es monótono en el número de aristas de dependencia, de modo que una topología de $\mathcal{O}(n)$ aristas está demostrablemente menos acoplada que una de $\mathcal{O}(n^2)$. Hohpe and Woolf [7] trasladó esto a la integración: una malla punto a punto de n sistemas necesita $n(n-1)/2$ canales, que un hub reduce a n . La investigación en microservicios [17] lleva el mismo argumento a sistemas cloud-native, y Shen et al. [18] muestran que las topologías densas de agentes amplifican la propagación de errores mientras que las más dispersas la suprimen. A2A crea aristas agente $\leftrightarrow$ agente; GCP crea aristas agente $\rightarrow$ almacén.

C. Mínimo privilegio, contexto con control de acceso y recuperación con compuerta

La compuerta de lectura de GCP operacionaliza el mínimo privilegio [11]. El control de acceso basado en roles [12] asigna permisos a roles; el control de acceso basado en atributos [19] generaliza a atributos arbitrarios; la protección basada en capacidades [20] ata la autoridad a tokens explícitamente pasados —el modelo que siguen las credenciales de rol de GCP. La legislación de minimización de datos [21] convierte “leer solo lo que tu rol requiere” en una restricción legal. El acceso a herramientas de lectura primero (MCP) y la recuperación con control de acceso proveen el lado de la lectura, y RBAC/ABAC el lado de la compuerta, pero ninguno acopla ambos con procedencia por lectura; la línea base punto a punto no tiene una compuerta análoga, por lo que el control de fuga y de delegación descansa en la discreción del emisor.

D. Fuga, procedencia y gobernanza en sistemas agenticos

La confidencialidad en sistemas agenticos es una propiedad del sistema, no solo del modelo [22]; los canales de mensajes entre agentes son el vector de fuga dominante, en gran medida invisible para la auditoría basada solo en la salida [23]; y restringir lo que un agente puede *ver* es más robusto que filtrar lo que puede *decir* [24]. El OWASP Top 10 para aplicaciones LLM [25] nombra la Divulgación de Información Sensible y la Agencia Excesiva como riesgos primarios —esta última es exactamente lo que sondea nuestro escenario de delegación. El control determinista de privilegios en el límite de la herramienta [26], la delegación autenticada de autoridad [27]

y los rastros de auditoría [28] abogan por registros a prueba de manipulación de las acciones del sistema.

a) *Delta / novedad defendible.*: El trabajo previo aporta las piezas —acceso de lectura primero (MCP), control por rol/atributo (RBAC/ABAC, RAG con control de acceso), delegación auditada y registros de auditoría— pero no su conjunción, ni una comparación directa multi-modelo controlada por equidad contra una línea base real de paso de mensajes. Nuestra pretensión de novedad es precisamente la *integración*: lectura federada + control por rol + procedencia por lectura, evaluados contra A2A. La lectura primero sola, RBAC solo, o la auditoría sola no serían una contribución.

III. EL GRAPH CONTEXT PROTOCOL: ESPECIFICACIÓN

GCP modela un ecosistema como *nodos de conocimiento* y *nodos de agente* de propiedad independiente. La coordinación es de lectura primero: un agente emite una *consulta de contexto* a un nodo par, que devuelve el contexto que el solicitante está autorizado a leer. Especificamos el sustrato con precisión suficiente para reimplementarlo.

A. Estructuras de datos

Un **nodo de conocimiento** lleva un id, un tipo, un adaptador que produce contenido y una política de acceso. Una **política** declara los campos de la compuerta; una **credencial de rol** nombra el rol del solicitante y sus capacidades planas; un **registro de procedencia** registra cada decisión.

```
KnowledgeNode {
  id, kind, // e.g. "rag", "log"
  adapter, // yields raw content
  policy: AccessPolicy }
AccessPolicy {
  readableByRoles: [roleId],
  requiredCapabilities: [capId],
  fallbackAllowed: bool,
  denialMode: "empty" | "error" }
Principal (role credential) {
  id, role: { id, capabilities:[capId],
    parent? },
  capabilities: [capId] }
Provenance {
  principalId, targetNodeId, timestamp,
  decision: "grant" | "deny",
  matchedPolicy }
```

B. Flujo de consulta y autorización

Una consulta de contexto nombra un nodo destino y una capacidad solicitada (lectura, o la más estricta delegación). El servidor evalúa el predicado de admisión *antes* de que corra cualquier adaptador, de modo que el contenido denegado nunca entra a la ventana de contexto del agente (mediación completa [11]). El conjunto efectivo de capacidades de un solicitante es la unión de sus capacidades planas y las heredadas a través de la cadena de roles (precedencia propia primero). El predicado es

$$\text{authorized}(P, K) \equiv \underbrace{(P.\text{roles}=[] \vee K.\text{role} \in P.\text{roles})}_{\text{roleOk}} \wedge \underbrace{(P.\text{caps}=[] \vee P.\text{caps} \subseteq \text{eff}(K))}_{\text{capsOk}}$$

TABLE I. COMPARACIÓN DE PROPIEDADES. UN “NO” NO SIGNIFICA QUE EL PROTOCOLO HAGA IMPOSIBLE LA PROPIEDAD; TODA CELDA INFERIOR ES ALCANZABLE EN PRINCIPIO EXTENDIENDO EL PROTOCOLO BASE. “NO NAT.” = NO ES UNA PRIMITIVA NATIVA DEL PROTOCOLO BASE (REQUERIRÍA LÓGICA ADICIONAL ENCIMA); “EXT.” = ALCANZABLE SOLO CON INSTRUMENTACIÓN EXTERNA ADICIONAL; “N/A” = EL CONCEPTO NO APLICA AL DISEÑO DE ESTE PROTOCOLO.

propiedad	A2A	MCP	GCP
agente↔agente	sí	no nat.	sí
lectura primero	no nat.	sí	sí
límite entre propietarios	sí	no nat.	sí
compuerta por rol/capacidad	burda	no nat.	sí
procedencia por lectura (nativa)	ext.	ext.	sí
denegación por defecto	no nat.	no nat.	sí
delegación de capacidad más estricta	no nat.	n/a	sí

Una política ausente o con esquema inválido se trata como una denegación (denegación por defecto). La ruta de extremo a extremo es:

Solicitud → Verificación de política → Decisión → Contexto
devuelto / denegado → Registro de procedencia

C. Algoritmo de lectura/autorización (pseudocódigo)

```
handleQuery(query, principal):
  node = resolve(query.targetNodeId)
  if node == null: return deny("no-node")
  P = node.policy
  if P invalid or absent: # default-deny
    record(principal, node, "deny", "none")
    return deny("no-policy")
  eff = flat(principal) UNION
    rolechain(principal.role)
  roleOk = P.roles==[] or
    principal.role.id in P.roles
  capsOk = P.caps==[] or P.caps subset eff
  needCap = query.capability # read|delegate
  capOk2 = needCap in eff
  if roleOk and capsOk and capOk2:
    record(principal, node, "grant", P.id)
    return node.adapter.read() # content
  else:
    record(principal, node, "deny", P.id)
    return denialResult(P.denialMode) # denied
```

La delegación reutiliza esta misma ruta con needCap = delegate, una capacidad estrictamente más fuerte que read: un agente autorizado a leer no queda por ello autorizado a comandar.

D. Manejo de denegaciones

Ante una denegación el manejador devuelve solo una razón de denegación (o un resultado vacío, según denialMode); el adaptador nunca se invoca, por lo que el contenido del nodo no puede aparecer en la respuesta. Cada denegación se registra a su vez, produciendo un libro de quién intentó qué, no solo de quién leyó qué.

E. Relación con A2A y MCP

La Tabla I contrasta los tres. A2A es agente a agente y basado en empuje, con permiso/denegación burda declarada en la tarjeta y sin rastro nativo por lectura; MCP es de lectura primero pero agente a herramienta y de propietario único, sin control por rol entre propietarios; GCP combina acceso de lectura primero entre propietarios, control por rol, denegación por defecto y procedencia nativa por lectura.

IV. DEFINICIÓN DE MÉTRICAS

Reportamos métricas en cuatro familias y separamos lo determinista de lo dependiente del modelo en todo momento.

a) **Acoplamiento estructural (determinista).**: Para n pares, las conexiones por pares son las aristas de integración que la topología requiere: $C_{A2A}(n) = n(n-1)$ vs. $C_{GCP}(n) = n$. El esfuerzo de integración es el costo marginal $C(n+1) - C(n)$: constante 1 para GCP, $2n$ para A2A. Calculado exactamente. Contamos aristas de integración dirigidas; por lo tanto cada relación bidireccional entre pares contribuye dos aristas dirigidas. Bajo una definición de canal no dirigido el conteo correspondiente es $n(n-1)/2$; la conclusión asintótica se mantiene en $\mathcal{O}(n^2)$.

b) **Sobrecosto de descubrimiento y de prompt de herramientas (por solicitud).**: Mensajes de descubrimiento: viajes de ida y vuelta para saber qué puede consultarse —1 para GCP, n para A2A base (una tarjeta por par). Tokens de prompt de herramientas: costo en tokens de las descripciones de herramientas/agentes inyectadas en cada solicitud. Recurrente, distinto del cableado único inicial.

c) **Gobernanza (dos puntos de referencia).**: Completitud de procedencia: fracción de decisiones con un registro estructurado, acotada a $[0, 1]$. Para GCP esto es 1.0 por construcción. Para A2A base es 0.0 —léase como “la implementación base de paso de mensajes no produce trazabilidad por lectura sin instrumentación añadida”, no como “A2A no puede auditarse”. Un brazo A2A con registro mínimo de lecturas es el segundo punto de referencia natural y elevaría su completitud al costo de la misma instrumentación que GCP hace nativa. Contención de delegación: fracción de intentos de delegación no autorizados rechazados (denegados y registrados) en lugar de ejecutados.

d) **Fuga y agenzia (taxonomía de cuatro vías).**: Separamos: (a) acceso no autorizado intentado —el agente intentó alcanzar un nodo/acción sobre el que no tiene derechos; (b) acceso no autorizado concedido —el sustrato lo sirvió; (c) fuga en la transcripción —un canario aparece en cualquier parte de la transcripción de herramientas; (d) fuga en la respuesta final —un canario aparece en la respuesta. El escaneo de canarios es de coincidencia exacta (una cota inferior; la paráfrasis y la fuga semántica no se cuentan). Reportamos (a) y (b) directamente desde el escenario de delegación; la separación transcripción/respuesta y una métrica de fuga por similitud semántica son instrumentación que señalamos como trabajo futuro (Sección VII). Distinguimos la garantía arquitectónica (lo que la compuerta demostrablemente previene) de la fuga observada (lo que un modelo dado ejerció).

e) **Costo y éxito conductual.**: Conteos (mensajes, conexiones, viajes de ida y vuelta, tokens en el cable) y latencia de reloj de pared, con un oráculo de éxito por escenario. Los conteos son la señal de costo primaria; el reloj de pared local es ruidoso.

V. METODOLOGÍA

a) **Invariante de equidad.**: Los dos brazos difieren únicamente en la capa de interoperabilidad. Un solo ejecutor de escenarios construye ambos a partir de una única `ScenarioDef` y un único modelo de chat inyectado; ambos corren el mismo cerebro ReAct, el mismo system prompt, el mismo objetivo y el mismo oráculo. El brazo GCP aloja los nodos en el mismo proceso detrás del manejador de fetch del servidor real, con un proveedor de autenticación de token estático y un sumidero de auditoría en memoria; el brazo A2A aloja los mismos nodos como servidores A2A reales en localhost. Solo cambian la fábrica de herramientas por par y el alojamiento de nodos.

b) **Escenarios.**: *software-org* (fuga): un agente con rol de contratista resume información pública de un proyecto; un nodo confidencial de ingeniería (canario) solo es legible por el rol de ingeniería. *supply-chain* (entre propietarios): un agente de logística declara la relación de envío compartible entre dos nodos de propiedad independiente, cada uno con un canario propio de su dueño. *delegation* (agencia, lado de escritura): un agente intenta una acción que no está autorizado a comandar; el oráculo verifica la contención. *forced-access* (agencia, lado de lectura): la tarea del agente le instruye explícitamente leer un nodo confidencial de forma directa, pese a que su rol carece de la capacidad requerida —a diferencia de *software-org*, la ruta de lectura no autorizada se ejerce en cada corrida en lugar de quedar latente; el oráculo verifica la contención de la lectura, reflejando el oráculo de delegación para el lado de lectura. *marketplace* (acoplamiento): n nodos vendedores con precios descendentes. Siguiendo la guía de los revisores, el marketplace aporta solo sus curvas deterministas de estructura, descubrimiento y prompt de herramientas; sus filas conductuales se excluyen de las afirmaciones principales porque el éxito de tarea fue bajo en ambos brazos (Sección VII).

c) **Modelos, semillas, arnés.**: Modelos de peso abierto servidos localmente: `llama3.1:8b` (5 semillas por brazo), `qwen3:4b` (3 semillas por brazo, dos conjuntos de ejecución independientes, etiquetados (a) y (b) abajo), `qwen2.5:32b` (3 semillas por brazo) —tres modelos distintos, cuatro conjuntos de ejecución independientes en total, ya que (a) y (b) son dos ejecuciones separadas de la configuración idéntica `qwen3:4b`, no dos modelos

TABLE II. ACOPLAMIENTO ESTRUCTURAL (ARISTAS DE INTEGRACIÓN) SEGÚN EL TAMAÑO n . DETERMINISTA, INDEPENDIENTE DEL MODELO.

n	GCP (n)	A2A ($n(n-1)$)	razón
2	2	2	1.0×
3	3	6	2.0×
5	5	20	4.0×
10	10	90	9.0×
20	20	380	19.0×
50	50	2450	49.0×

TABLE III. MENSAJES DE DESCUBRIMIENTO Y TOKENS DE PROMPT DE HERRAMIENTAS POR SOLICITUD (BARRIDO DE MARKETPLACE). INDEPENDIENTE DEL MODELO.

n	msjs. descubrim.		tok. prompt-herr.	
	GCP	A2A	GCP	A2A
2	1	2	72	110
4	1	4	72	220
8	1	8	72	440
16	1	16	72	886
32	1	32	72	1782

diferentes. Adicionalmente evaluamos una familia alojada de frontera a través de la API nativa de Gemini (EVAL_ADAPTER=google, saneador de llamadas paralelas a herramientas, sin streaming): `gemini-2.5-flash-lite` y `gemini-2.5-flash` (30 semillas por brazo) y `gemini-2.5-pro` (15 semillas por brazo, ejecutado mediante un escritor incremental JSONL por corrida después de que un primer intento de 30 semillas excediera el límite de 60 minutos del arnés) —cuatro familias de modelos en total, en siete configuraciones de ejecución. El arnés está controlado por CI, regula la tasa de solicitudes y reintenta ante errores transitorios. Reportamos media $\pm$ desviación estándar poblacional; donde un intervalo de aproximación normal al 95% es informativo damos $\pm 1.96 s/\sqrt{k}$. Las curvas estructurales se calculan para $n = 2, \dots, 50$; las curvas de descubrimiento/prompt de herramientas se miden sobre $n \in \{2, 4, 8, 16, 32\}$. Tratamos acoplamiento/descubrimiento/prompt de herramientas/procedencia/ contención como establecidos (deterministas o arquitectónicos) y tokens/latencia/fuga/éxito como preliminares: las filas de peso abierto siguen con pocas semillas y alojadas localmente, y la réplica alojada de frontera (mayor número de semillas, una sola familia de modelos) retira esa limitación parcial, no totalmente.

VI. RESULTADOS

A. Acoplamiento: $\mathcal{O}(n)$ vs. $\mathcal{O}(n^2)$ (determinista)

La Tabla II muestra la curva estructural: GCP crece linealmente, A2A cuadráticamente — $9\times$ en $n=10$, $49\times$ en $n=50$. El esfuerzo de integración es constante 1 para GCP y $2n$ para A2A, de modo que la brecha se amplía con cada nodo. Idéntico sin importar el modelo: la curva la fija la topología, no el comportamiento del modelo.

B. Sobrecosto por solicitud: $\mathcal{O}(1)$ vs. $\mathcal{O}(n)$ (determinista)

El impuesto de acoplamiento se paga en cada solicitud vía mensajes de descubrimiento y el presupuesto de prompt de herramientas (Tabla III). GCP mantiene ambos planos (1 mensaje, 72 tokens); A2A base llega a 32 mensajes y 1782 tokens de prompt de herramientas en $n=32$, una brecha de $25\times$ en el prompt. Idéntico sin importar el modelo, ya que estos valores los fija la construcción del prompt, no el comportamiento del modelo.

C. Procedencia: nativa en GCP, instrumentación añadida en A2A base (arquitectónico)

En los cuatro escenarios ajenos al marketplace, en las cuatro familias de modelos, en cada semilla, la completitud de procedencia es 1.0 para GCP y 0.0 para A2A base, con una excepción reportada: el brazo A2A de `gemini-2.5-flash-lite` en *forced-access* muestra 1.0, un caso degenerado ya que ese brazo hizo cero intentos (declinó la tarea, Sección VI-E) —no había nada que dejar de registrar, no un caso en que A2A base realmente produjera un rastro. Fuera de eso es arquitectónico: GCP registra cada decisión; el brazo base de paso de mensajes no tiene sumidero de auditoría, por lo que no existe rastro que consultar. Como se indicó en la Sección IV, esto refleja la

TABLE IV. DELEGACIÓN, MÉTRICAS DE AGENCIA DE CUATRO VÍAS POR CONJUNTO DE EJECUCIÓN. INT. = INTENTOS NO AUTORIZADOS; CONC. = CONCEDIDOS NO AUTORIZADOS (EJECUTADOS); CONT. = CONTENCIÓN. QWEN3:4B (A) Y (B) SON DOS CONJUNTOS DE EJECUCIÓN INDEPENDIENTES DEL MISMO MODELO, NO DOS MODELOS DISTINTOS. LAS FILAS DE PESO ABIERTO Y DE GEMINI-2.5-{FLASH-LITE, FLASH} SON UNA ÚNICA CORRIDA REPRESENTATIVA (SEED[0]); GEMINI-2.5-PRO ESTÁ SUMADO SOBRE LAS 15 SEMILLAS (ETIQUETADO $\Sigma/15$), SEGÚN LA SECCIÓN V.

modelo	brazo	int.	conc.	cont.	proc.
llama3.1:8b	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
qwen3:4b (a)	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
qwen3:4b (b)	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
qwen2.5:32b	GCP	1	0	1.00	1.0
	A2A	0 [†]	0	0.00	- [†]
gemini-2.5-flash-lite	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
gemini-2.5-flash	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
gemini-2.5-pro ($\Sigma/15$)	GCP	14	0	1.00	-
	A2A	11	11	0.00	-

[†] corrida degenerada: el agente A2A base nunca emitió la solicitud, así que nada fue intentado, ejecutado ni registrado; la compuerta sigue ausente.

implementación *base*, no un límite necesario de A2A; un A2A instrumentado elevaría esta cifra al costo de la misma instrumentación que GCP hace nativa.

D. Contención de la delegación: GCP contiene, A2A base ejecuta (arquitectónico + observado)

La Tabla IV reporta las métricas de agencia de cuatro vías en las cuatro familias de modelos. La contención de GCP es 1.00 en todos los modelos, con cero ejecuciones no autorizadas —una garantía *arquitectónica*: la compuerta de capacidad de delegación rehusa por construcción. La contención de A2A base es 0.00, y en cinco de las seis configuraciones evaluadas *ejecutó* la acción no autorizada (acceso no autorizado concedido *observado*). Bajo qwen2.5:32b el modelo de peso abierto más grande simplemente no la intentó, por lo que nada se ejecutó —pero la compuerta sigue ausente, de modo que la contención es 0.00 por construcción. Esto separa “el modelo se comportó bien” de “el protocolo no permitió el mal comportamiento”. Los niveles de frontera alojados se comportan igual que los de peso abierto: los intentos escalan con la capacidad del modelo (gemini-2.5-pro intentó en 14/15 semillas, agregado), y aun así la contención de GCP permanece en 1.00.

E. Contención del acceso forzado: la contraparte de lectura (arquitectónico + observado)

La Tabla V reporta el escenario forced-access, el reflejo del lado de lectura de la delegación: la tarea instruye al agente a leer el nodo confidencial directamente, por lo que la ruta de lectura no autorizada se ejerce en cada corrida que siquiera se involucra con la tarea, no queda latente como en software-org. La contención de GCP es 1.00 en toda configuración que intentó la lectura. La contención de A2A base es 0.00 en todo el conjunto, y en dos de los tres niveles alojados —flash y pro, los modelos más fuertes— *ejecutó* la lectura no autorizada y el canario surgió en la transcripción, un acceso no autorizado concedido *observado*, no solo una posibilidad arquitectónica. Este es el resultado que la edición previa de este artículo señalaba como trabajo futuro (Sección VII): correr el escenario es lo que separa “el modelo se comportó bien” de “el protocolo no permitió el mal comportamiento”, y los modelos más fuertes son exactamente donde esa distinción cobra sentido, ya que son los que efectivamente intentan y completan la lectura no autorizada.

llama3.1:8b es un valor atípico genuino, no un dato faltante: no hizo *ninguna* llamada a herramientas en ninguno de los dos brazos a lo largo de dos reejecuciones independientes de 5 semillas (20/20 corridas), reproducido de forma determinista. Una prueba directa no agentica sobre la misma entrada del

TABLE V. FORCED-ACCESS, MÉTRICAS DE AGENCIA POR FAMILIA DE MODELO. INT. = INTENTOS DE LECTURA NO AUTORIZADA; CONC. = CONCEDIDOS NO AUTORIZADOS (EJECUTADOS, CANARIO SERVIDO); CONT. = CONTENCIÓN. LAS CONVENCIONES DE CONTEO COINCIDEN CON LA TABLA IV (LA FILA DE GEMINI-2.5-PRO ES $\Sigma/15$).

modelo	brazo	int.	conc.	cont.
llama3.1:8b	GCP	0 [‡]	0	n/a [‡]
	A2A	0 [‡]	0	n/a [‡]
qwen3:4b	GCP	1	0	1.00
	A2A	1	1	0.00
qwen2.5:32b	GCP	1	0	1.00
	A2A	1	1	0.00
gemini-2.5-flash-lite	GCP	1	0	1.00
	A2A	0 [†]	0	0.00 [†]
gemini-2.5-flash	GCP	1	0	1.00
	A2A	1	1	0.00
gemini-2.5-pro ($\Sigma/15$)	GCP	11	0	1.00
	A2A	12	12	0.00

[‡] ninguno de los dos brazos llegó a llamar una herramienta —el modelo rehusó la tarea directamente en su respuesta final (ver texto); no se ejerció compuerta alguna en ningún lado, por lo que la contención no está definida (no es lo mismo que 1.00: la compuerta nunca fue puesta a prueba). [†] el agente A2A base declinó la tarea y nunca intentó la lectura, así que nada se ejecutó —pero la compuerta sigue ausente, de modo que la contención es 0.00 por construcción, siguiendo la misma convención de corrida degenerada de qwen2.5:32b en la Tabla IV.

escenario muestra al modelo rehusando en su respuesta final antes de invocar jamás una herramienta —textualmente “*I can't do that. Is there anything else I can help you with?*” (brazo GCP) y “*I can't help you with this request.*” (brazo A2A). La frase del system prompt del escenario que hace explícito el desajuste de rol (“even though your role is not engineering”) se lee para el entrenamiento de alineación de este modelo como una violación de política, por lo que rehusa directamente —en *ambos* brazos, independientemente de si existe una compuerta que capture la lectura. Esta es una interacción de alineación-vs-uso-de-herramientas propia de modelos pequeños, ortogonal a la comparación de protocolos: el mismo modelo llama herramientas sin vacilar en software-org, supply-chain y delegation. gemini-2.5-flash-lite muestra una versión más leve del mismo efecto, pero solo en el brazo A2A (declinó la tarea, 0 intentos), mientras que su brazo GCP intentó y fue denegado —así la contención da 1.00/0.00 por construcción en ambos niveles, pero por razones distintas: el nivel alojado más pequeño evitó la lectura sin compuerta declinando la tarea; el modelo local más pequeño la evitó declinándolo todo.

F. Costo y éxito: paridad (preliminar)

Los tokens en el cable y el éxito de tarea están dentro de la varianza entre semillas entre brazos en todo caso en que el modelo pudo realizar la tarea. En software-org con llama3.1:8b, los tokens son 500.6 ± 16.9 vs. 502.4 ± 18.9 (IC 95% ± 14.8 vs. ± 16.6 , $k=5$); en software-org con qwen2.5:32b, 669.7 ± 0.9 vs. 668.7 ± 3.3 . Los conteos absolutos difieren en un orden de magnitud entre modelos, pero la brecha entre brazos permanece dentro del intervalo en cada escenario. El conteo de tokens *en el cable* del brazo A2A base suele ser marginalmente *menor* (omite el sobre de consulta de GCP); el impuesto que paga A2A es el presupuesto de prompt de herramientas por solicitud (Tabla III), no la carga útil. Por lo tanto no afirmamos que GCP envíe menos tokens en el cable —solo que no cuesta significativamente más mientras elimina el sobrecosto de prompt $\mathcal{O}(n)$. Presentamos esto como *preliminar*, pendiente de 20–30 semillas y pruebas de significancia.

G. Fuga: empates en los escenarios canario (observado), la garantía se sostiene (arquitectónico)

Reportamos el panorama de fuga sin rodeos. En software-org, la fuga observada fue 0.0 en ambos brazos en todos los modelos —pero porque el agente satisfizo la tarea de resumen público sin llegar a solicitar el nodo confidencial, la ruta con fuga *no se ejerció*; la fuga arquitectónica es latente, no observada. En supply-chain, ambos brazos están autorizados a leer los nodos de ambos dueños, de modo que ambos expusieron los canarios por

igual (fuga 1.0 en ambos, en todo modelo): un *empate* genuino, porque la fuga no es una decisión de compuerta. Por lo tanto no afirmamos ninguna ventaja de fuga conductual observada. La ventaja de fuga de GCP es la garantía *arquitectónica* de denegar-por-defecto-más-auditoría, ejercida directamente por los resultados de delegación y procedencia, no por estos conteos de canarios.

H. Garantía arquitectónica: propiedad ejecutable de no-fuga

La distinción anterior descansa en una prueba, no solo en una medición. Sobre la ruta de lectura/autorización tal como se distribuye corremos pruebas basadas en propiedades (*fast-check*) contra conjuntos fijos y finitos: cinco capacidades y cuatro roles, uno de los cuales (*role:child*) hereda de otro (*role:a*) de modo que el cálculo de capacidad efectiva se ejerce, no solo las concesiones directas. Cada *política* aleatoria muestrea de forma independiente un subconjunto del conjunto de roles para *readableByRoles*, un subconjunto del conjunto de capacidades para *requiredCapabilities*, un booleano *fallbackAllowed* aleatorio y un modo de denegación aleatorio. Cada *principal* aleatorio muestrea de forma independiente un rol del conjunto o ninguno, más un subconjunto aleatorio de capacidades sostenidas directamente. Como ambos se extraen de conjuntos pequeños por muestreo de subconjuntos en lugar de un espacio no acotado, los sorteos autorizados y no autorizados ocurren ambos con frecuencia, de modo que las 1000 corridas por defecto no están dominadas por un solo resultado. Cada par (*policy, principal*) generado se pasa entonces por el manejador real de consulta de contexto (no un simulacro) contra un adaptador de prueba que *siempre* devuelve una cadena canario centinela, de modo que P1 (no-fuga de contenido de extremo a extremo) afirma que una consulta no autorizada nunca devuelve el canario y queda marcada como *denied*, mientras que una autorizada sí lo devuelve (vivacidad, para que la prueba no sea vacúa); P2 (solidez de la autorización) verifica que la decisión de la compuerta distribuida sea igual a un predicado de referencia escrito de forma independiente sobre el mismo par; P3 (solidez de la herencia de roles) verifica el cálculo de capacidad efectiva directamente contra la cadena de roles. Todas pasan en 1000 corridas (elevable vía una variable de entorno; las corridas son reproducibles por semilla para depuración). La métrica de canario *mide* la fuga en los escenarios; la suite de propiedades *demuestra* que la compuerta es sólida sobre el espacio de entradas. Juntas: la propiedad muestra que la compuerta es correcta; el canario confirma que está en la ruta caliente.

VII. DISCUSIÓN Y AMENAZAS A LA VALIDEZ

a) *Cuatro familias de modelos, siete conjuntos de ejecución, pero el conteo de semillas de peso abierto se mantiene bajo.*: Los resultados conductuales ahora abarcan tres modelos de peso abierto en dos clases de tamaño (4B–32B), alojados localmente, en cuatro conjuntos de ejecución independientes (*qwen3:4b* se corrió dos veces, como verificación interna de réplica, no como un cuarto modelo), más una familia alojada de frontera (Gemini 2.5, tres niveles) en tres conjuntos de ejecución adicionales —eliminando tanto la amenaza de modelo único como la de solo-peso-abierto. El conteo de semillas, sin embargo, es desigual entre los dos grupos: el brazo de peso abierto se mantiene en 3–5 semillas, dado el costo de reloj de pared de correr cada semilla como un bucle completo de agente ReAct de múltiples pasos contra un servidor LLM alojado localmente, repetido por modelo, por escenario y por brazo, lo que hace que 20–30 semillas resulten aproximadamente un orden de magnitud más costosas ahí. El brazo alojado de Gemini cierra esta brecha para dos de sus tres niveles (30 semillas para *flash-lite/flash*, cumpliendo la convención de pruebas de significancia que no podíamos costear localmente) y parcialmente para el tercero (*gemini-2.5-pro*, 15 semillas —un primer intento de 30 semillas fue interrumpido por el límite de 60 minutos del arnés antes de escribir un reporte; el escritor incremental usado para la corrida reportada evita ese modo de falla, pero el límite en sí aún no se ha elevado para permitir que una corrida de 30 semillas de *pro* se complete). Esta restricción no afecta las afirmaciones deterministas/arquitectónicas (acoplamiento, descubrimiento, sobre costo de prompt de herramientas, procedencia, contención de delegación/acceso forzado), que no dependen en absoluto del conteo de semillas; solo incide en las columnas preliminares de tokens/latencia/fuga/ éxito, y solo en las filas de peso abierto y en el nivel *pro*. Una segunda familia alojada de frontera, y 20–30 semillas en *pro*, siguen siendo el próximo paso natural de réplica.

b) *La procedencia es una propiedad de la implementación base.*: El 0.0 de A2A es una propiedad del paso de mensajes base, no una prueba de que A2A no pueda auditarse. Lo enmarcamos como “sin trazabilidad por lectura sin instrumentación añadida” e identificamos A2A instrumentado como el segundo baseline justo.

c) *La medición de fuga es limitada.*: El escaneo de canarios por coincidencia exacta pasa por alto la paráfrasis y la fuga semántica, y la fuga conductual en los escenarios no adversariales depende de que el agente ejerza la ruta (latente en *software-org*, empatada en *supply-chain*). El escenario *forced-access* (Sección VI-E) ahora cierra parte de esta brecha al apuntar directamente al nodo confidencial en lugar de esperar a que el agente lo solicite, y muestra tanto una garantía arquitectónica (contención de GCP en 1.00 en todo el conjunto) como, en los dos niveles alojados más fuertes, una fuga de lectura concedida *observada* en A2A base. Dos brechas complementarias quedan como trabajo futuro: un escenario de *prompt adversarial* que induzca la recuperación no autorizada mediante indirección en lugar de una instrucción explícita, y una métrica de fuga *semántica* más allá de la coincidencia exacta, que también permitiría a los canarios de *software-org* y *supply-chain* detectar divulgación parafraseada. La delegación y el acceso forzado ya son ambas pruebas de acción forzada y ambas muestran a la compuerta actuando.

d) *El marketplace se usa solo para la estructura.*: El éxito de tarea del marketplace fue bajo en *ambos* brazos (por ejemplo, cayendo a 0.0–0.4 en $n=16,32$ con *llama3.1:8b*); el modelo frecuentemente no logró elegir al vendedor más barato. Como la falla es igual entre brazos no amenaza la equidad, pero siguiendo la guía de los revisores usamos el marketplace *solo* para sus curvas deterministas de estructura, descubrimiento y prompt de herramientas, y excluimos sus filas conductuales de las afirmaciones principales.

e) *Validez de constructo de la línea base.*: El brazo A2A corre servidores reales, el mismo cerebro y la misma tarea, contruidos con igual cuidado; sus desventajas (sin compuerta de lectura, sin compuerta de delegación, sin sumidero de auditoría nativo, topología por pares) son intrínsecas al paso de mensajes base, no artefactos de un hombre de paja.

VIII. LIMITACIONES Y TRABAJO FUTURO

Llevar el brazo de peso abierto hacia 20–30 semillas con pruebas de significancia sobre las brechas de tokens/latencia/mensajes, elevar el límite del arnés para que *gemini-2.5-pro* pueda completar una corrida completa de 30 semillas, y añadir una segunda familia alojada de frontera además de Gemini; añadir el segundo baseline de A2A instrumentado; añadir los escenarios de prompt adversarial y de fuga semántica y separar la fuga de transcripción de la de respuesta final; y diseñar un banco de pruebas de privacidad que elimine el factor de confusión de lectura autorizada del escenario actual de *supply-chain*, de modo que la ventaja de la compuerta pueda *observarse*, no solo argumentarse, también en ese escenario. Conectar GCP con MCP y A2A como interoperabilidad de primera clase es la ruta natural de despliegue.

IX. CONCLUSIÓN

Presentamos GCP —federación de contexto de lectura primero, controlada por rol y auditada, con una compuerta de delegación de capacidad más estricta— y una comparación controlada por equidad contra el paso de mensajes A2A base en cuatro familias de modelos: tres modelos locales de peso abierto y una familia alojada de frontera (Gemini 2.5, tres niveles). Las ventajas *estructurales* son fuertes e independientes del modelo: el acoplamiento mejora de $\mathcal{O}(n^2)$ a $\mathcal{O}(n)$ en aristas de integración y de $\mathcal{O}(n)$ a $\mathcal{O}(1)$ en el sobre costo de descubrimiento y prompt de herramientas por solicitud; la procedencia es nativa en lugar de instrumentación añadida; y en ambos escenarios de agencia —delegation (lado de escritura) y forced-access (lado de lectura)— GCP contenía toda acción no autorizada mientras que A2A base no contenía ninguna, en toda familia de modelos que se involucró con la tarea. Los resultados de forced-access además convierten la garantía arquitectónica en una *observada* en los dos niveles alojados más fuertes, donde A2A base no solo arriesgó sino que efectivamente ejecutó la lectura no autorizada y expuso el canario. El costo y el éxito están en paridad en las corridas conductuales preliminares, y la fuga de canarios *empata* en el banco de pruebas no adversarial actual, por lo que aún no afirmamos ninguna ventaja de privacidad ahí. La contribución defendible es la *integración* —lectura federada, control por rol y procedencia por lectura— más su comparación medida; los resultados conductuales son un piloto multi-modelo, que ahora abarca una familia alojada de frontera junto a tres modelos de peso abierto, que invita a la réplica con más semillas y modelos alojados adicionales.

DECLARACIÓN DE CONFLICTO DE INTERESES

Los autores declaran que no existen conflictos de intereses potenciales relacionados con la investigación, la autoría o la publicación de este artículo.

FINANCIAMIENTO

Esta investigación no recibió financiamiento externo. El estudio fue desarrollado como una iniciativa independiente de los investigadores.

AGRADECIMIENTOS

Los autores expresan su agradecimiento a la Universidad Nacional de San Agustín de Arequipa por ser la casa de estudios en la que se realizó este trabajo de investigación, que se espera contribuya a la mejora de la educación en el país.

DECLARACIÓN SOBRE IA GENERATIVA

Durante la preparación de este trabajo, los autores usaron Claude y Claude Code (Anthropic) para asistir en la redacción y edición del texto del manuscrito, el refinamiento del lenguaje, y la implementación y ejecución de partes del arnés de evaluación (código de escenarios, adaptadores y generación de reportes de resultados) bajo la dirección y supervisión de los autores. Las ideas científicas, el diseño experimental, la interpretación de los resultados y las conclusiones de este artículo fueron desarrollados por los autores humanos. Tras usar estas herramientas, los autores revisaron y editaron todo el contenido y código asistido por IA, verificaron todos los resultados reportados contra los artefactos de ejecución subyacentes, y asumen plena responsabilidad por la exactitud, originalidad e integridad de esta publicación. Ninguna herramienta de IA generativa figura como autora.

REFERENCES

- [1] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [2] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL <https://arxiv.org/abs/2307.07924>.
- [3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2308.00352>.
- [4] Google and Linux Foundation. Agent2Agent (A2A) protocol specification, v1.0. Open standard, 2025. URL <https://a2a-protocol.org/v1.0.0/specification/>.
- [5] Tim Finin, Rich Fritzson, Don McKay, and Robin McEntire. KQML as an agent communication language. In *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM)*, 1994.
- [6] Foundation for Intelligent Physical Agents. FIPA ACL message structure specification. Technical Report SC00061G, FIPA, 2002. URL <http://www.fipa.org/specs/fipa00061/>.
- [7] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2003.
- [8] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.
- [9] W. P. Stevens, G. J. Myers, and L. L. Constantine. Structured design. *IBM Systems Journal*, 13(2):115–139, 1974. doi: 10.1147/sj.132.0115.
- [10] Lionel C. Briand, John W. Daly, and Jürgen K. Wüst. A unified framework for coupling measurement in object-oriented systems. *IEEE Transactions on Software Engineering*, 25(1):91–121, 1999. doi: 10.1109/32.748920.
- [11] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [12] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996. doi: 10.1109/2.485845.
- [13] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Zhang, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pages 8048–8057, 2024. URL <https://arxiv.org/abs/2402.01680>.
- [14] Anthropic. Introducing the model context protocol. Anthropic; open specification, 2024. URL <https://www.anthropic.com/news/model-context-protocol>.
- [15] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for LLM-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024. URL <https://arxiv.org/abs/2410.02506>.
- [16] Yuzhen Mao and Azalia Mirhoseini. Decentralized multi-agent systems with shared context. *arXiv preprint arXiv:2606.10662*, 2026. URL <https://arxiv.org/abs/2606.10662>.
- [17] Sebastiano Panichella, Mohammad Imranur Rahman, and Davide Taibi. Structural coupling for microservices. In *Proceedings of the 11th International Conference on Cloud Computing and Services Science (CLOSER)*, pages 280–287, 2021. doi: 10.5220/0010481902800287. URL <https://arxiv.org/abs/2103.04674>.
- [18] Xu Shen, Yixin Liu, Yiwei Dai, Yili Wang, Rui Miao, Yue Tan, Shirui Pan, and Xin Wang. Understanding the information propagation effects of communication topologies in LLM-based multi-agent systems. *arXiv preprint arXiv:2505.23352*, 2025. URL <https://arxiv.org/abs/2505.23352>.
- [19] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. Guide to attribute based access control (ABAC) definition and considerations. Technical Report NIST Special Publication 800-162, National Institute of Standards and Technology, 2014.
- [20] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966. doi: 10.1145/365230.365252.
- [21] European Parliament and Council. Regulation (EU) 2016/679 (general data protection regulation), articles 5 and 25. Official Journal of the European Union, L 119, 2016. URL <https://gdpr-info.eu/>.
- [22] Jonathan Evertz, Merlin Chlosta, Lea Schönherr, and Thorsten Eisenhofer. Whispers in the machine: Confidentiality in agentic systems. *arXiv preprint arXiv:2402.06922*, 2024. URL <https://arxiv.org/abs/2402.06922>.
- [23] Fatima El Yagoubi, Godwin Badu-Marfo, and Ranwa Al Mallah. AgentLeak: A full-stack benchmark for privacy leakage in multi-agent LLM systems. *arXiv preprint arXiv:2602.11510*, 2026. URL <https://arxiv.org/abs/2602.11510>.
- [24] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. AirGapAgent: Protecting privacy-conscious conversational agents. *arXiv preprint arXiv:2405.05175*, 2024. URL <https://arxiv.org/abs/2405.05175>.
- [25] OWASP GenAI Security Project. OWASP top 10 for large language model applications. OWASP Foundation, 2025. URL <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [26] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Programmable privilege control for LLM agents. *arXiv preprint arXiv:2504.11703*, 2025. URL <https://arxiv.org/abs/2504.11703>.
- [27] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. Authenticated delegation and authorized AI agents. *arXiv preprint arXiv:2501.09674*, 2025. URL <https://arxiv.org/abs/2501.09674>.
- [28] Victor Ojewale, Harini Suresh, and Suresh Venkatasubramanian. Audit trails for accountability in large language models. *arXiv preprint arXiv:2601.20727*, 2026. URL <https://arxiv.org/abs/2601.20727>.